

Knowledge Quiz 2

Cynthia Budiyanto

Guidelines:

- No consulting with anyone else (other than clarification questions from Professor Roith)
- You may use only materials from this class (our class webpage, links on Moodle, our 3 online textbooks, files posted to the RStudio server)
- No online searches or use of large language models like ChatGPT

Pledge:

I pledge my honor that on this quiz I have neither given nor received assistance not explicitly approved by the professor and that I am aware of no dishonest work.

- type your name here to acknowledge the pledge: Cynthia Budiyanto

```
library(tidyverse)
library(nycflights13)
```

1. Here's is a crazy list that tells you some stuff about data science. Give code that will produce **exactly** the following outputs. (There are four parts to this question.)

```
kq2_data <- list(
  DRY = c("Do not", "Repeat", "Yourself"),
  knowledge = c("Knowledge", "is", "Power"),
  nyc = flights,
  integers = 1:25,
  doubles = sqrt(1:25),
  toppings = list(sauce = c("marinara", "bbq", "creamy garlic"),
    cheese = c("mozzarella", "goat", "feta"),
    meat = c("pepperoni", "sausage", "chicken", "none"),
    veggie = c("peppers", "onions", "mushrooms", "pineapple"),
    hold = "no"),
  lostpts_to_perc = cbind(pts_lost = 0:50,
```

```

    perc = (50 - 0:50) / 50)
)

str(kq2_data)

```

List of 7

```

$ DRY      : chr [1:3] "Do not" "Repeat" "Yourself"
$ knowledge : chr [1:3] "Knowledge" "is" "Power"
$ nyc      : tibble [336,776 x 19] (S3: tbl_df/tbl/data.frame)
..$ year    : int [1:336776] 2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
..$ month   : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
..$ day     : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
..$ dep_time : int [1:336776] 517 533 542 544 554 554 555 557 557 558 ...
..$ sched_dep_time: int [1:336776] 515 529 540 545 600 558 600 600 600 600 ...
..$ dep_delay : num [1:336776] 2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
..$ arr_time  : int [1:336776] 830 850 923 1004 812 740 913 709 838 753 ...
..$ sched_arr_time: int [1:336776] 819 830 850 1022 837 728 854 723 846 745 ...
..$ arr_delay : num [1:336776] 11 20 33 -18 -25 12 19 -14 -8 8 ...
..$ carrier   : chr [1:336776] "UA" "UA" "AA" "B6" ...
..$ flight    : int [1:336776] 1545 1714 1141 725 461 1696 507 5708 79 301 ...
..$ tailnum   : chr [1:336776] "N14228" "N24211" "N619AA" "N804JB" ...
..$ origin    : chr [1:336776] "EWR" "LGA" "JFK" "JFK" ...
..$ dest      : chr [1:336776] "IAH" "IAH" "MIA" "BQN" ...
..$ air_time  : num [1:336776] 227 227 160 183 116 150 158 53 140 138 ...
..$ distance  : num [1:336776] 1400 1416 1089 1576 762 ...
..$ hour      : num [1:336776] 5 5 5 5 6 5 6 6 6 6 ...
..$ minute    : num [1:336776] 15 29 40 45 0 58 0 0 0 0 ...
..$ time_hour : POSIXct[1:336776], format: "2013-01-01 05:00:00" "2013-
01-01 05:00:00" ...
$ integers   : int [1:25] 1 2 3 4 5 6 7 8 9 10 ...
$ doubles    : num [1:25] 1 1.41 1.73 2 2.24 ...
$ toppings   :List of 5
..$ sauce    : chr [1:3] "marinara" "bbq" "creamy garlic"
..$ cheese   : chr [1:3] "mozzarella" "goat" "feta"
..$ meat     : chr [1:4] "pepperoni" "sausage" "chicken" "none"
..$ veggie   : chr [1:4] "peppers" "onions" "mushrooms" "pineapple"
..$ hold     : chr "no"
$ lostpts_to_perc: num [1:51, 1:2] 0 1 2 3 4 5 6 7 8 9 ...
..- attr(*, "dimnames")=List of 2
.. ..$ : NULL
.. ..$ : chr [1:2] "pts_lost" "perc"

```

a)

```
[1] "Power" "is" "Knowledge"
```

```
kq2_data[[2]][c(3,2,1)]
```

```
[1] "Power"      "is"         "Knowledge"
```

b)

A tibble: 6 × 3

Groups: dest [1]

```
dest carrier mean_arr_delay 1 MSP 9E 8.09 2 MSP DL 4.04 3 MSP EV 10.5 4 MSP
MQ 9.56 5 MSP OO -2
6 MSP UA -5.5
```

```
kq2_data[[3]] |>
  select(dest, carrier, arr_delay) |>
  filter(dest == "MSP") |>
  filter(!is.na(arr_delay)) |>
  group_by(dest, carrier) |>
  summarise(mean_arr_delay = mean(arr_delay))
```

`summarise()` has grouped output by 'dest'. You can override using the
`.groups` argument.

```
# A tibble: 6 x 3
# Groups:   dest [1]
  dest carrier mean_arr_delay
  <chr> <chr>          <dbl>
1 MSP   9E              8.09
2 MSP   DL              4.04
3 MSP   EV             10.5
4 MSP   MQ              9.56
5 MSP   OO              -2
6 MSP   UA             -5.5
```

- c) Note: It must be one string and you must subset from kq2_data for the words “bbq”, “goat”, “chicken”, and “mushroom”

[1] “Give me a bbq pizza with goat cheese, chicken, and mushrooms.”

```
# Try 1

b <- kq2_data[[6]][[1]][2] #bbq

g <- kq2_data[[6]][[2]][2] #goat

c <- kq2_data[[6]][[3]][3] #chicken

m <- kq2_data[[6]][[4]][3] #mushrooms

str_glue(
  "[1] ",
  "\"",
  "Give me a {b} ",
  "pizza with {g} ",
  "cheese, {c}",
  ", and {m}",
  ".\""
)
```

[1] "Give me a bbq pizza with goat cheese, chicken, and mushrooms."

```
#Try 2

mystrings <- c(f1 = "Give me a", f2 = "pizza with", f3 = "cheese,", f4 = ", and")

answer <- c(mystrings[1], kq2_data[[6]][[1]][2], mystrings[2], kq2_data[[6]][[2]][2], mystrings[3], kq2_data[[6]][[3]][3], mystrings[4], kq2_data[[6]][[4]][3])
```

d)

[1] 10.0 0.8

```
as.numeric(kq2_data[[7]][11,])
```

[1] 10.0 0.8

2. Write a function called `test_genre_popularity()` that produces a single ggplot. It should allow a user to input:

- 1) a tibble
- 2) two song genres
- 3) and a plot option to create either a boxplot or faceted histograms

The title of your plot should indicate the two genres being compared. done

The subtitle should provide the *unique* test statistic and p-value for the specific genres being used, these values should change in the subtitle when you use different genres in the function input

(Hint: the `t.test()` function will automatically calculate these numbers).

Finally, your function should return an error when the user provides any plot option other than “boxplot” or “histogram”. done

See the image KQ2_plot.png for examples of what the function should output.

```
bigspotify <- readr::read_csv('https://raw.githubusercontent.com/rfordatascience/tidytuesday,
```

```
Rows: 32833 Columns: 23
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (10): track_id, track_name, track_artist, track_album_id, track_album_na...
```

```
dbl (13): track_popularity, danceability, energy, key, loudness, mode, spec...
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
bigspotify
```

```
# A tibble: 32,833 x 23
```

	track_id	track_name	track_artist	track_popularity	track_album_id
	<chr>	<chr>	<chr>	<dbl>	<chr>
1	6f807x0ima9a1j3VPbc7~	I Don't C~	Ed Sheeran	66	2oCs0DGTsR098~
2	0r7CVbZTWZgbTCYdfa2P~	Memories ~	Maroon 5	67	63rPS0264uRjW~
3	1z1Hg7Vb0AhHDIEmnDE7~	All the T~	Zara Larsson	70	1HoSmj2eLcsrR~
4	75FpbthrwQmzHlBJLuGd~	Call You ~	The Chainsm~	60	1nqYsOeflyKKu~
5	1e8PAfckUYoKkxPhrHqw~	Someone Y~	Lewis Capal~	69	7m7vv9wlQ4i0L~
6	7fvUMiyapMsRRxr07cU8~	Beautiful~	Ed Sheeran	67	2yiy9cd2QktrN~
7	20Ay1PUDDfwRGfe01Yql~	Never Rea~	Katy Perry	62	7INHYSeusaFly~
8	6b1RNvAcJjQH73eZ04BL~	Post Malo~	Sam Feldt	69	6703SRPsLkS4b~
9	7bF6tC03gFb8INrEDcjN~	Tough Lov~	Avicii	68	7CvAfGvq4RlIw~
10	1IXGILkPm0tOCNeq00kC~	If I Can'~	Shawn Mendes	67	4QxzbfSsVryEQ~

```
# i 32,823 more rows
# i 18 more variables: track_album_name <chr>, track_album_release_date <chr>,
#   playlist_name <chr>, playlist_id <chr>, playlist_genre <chr>,
#   playlist_subgenre <chr>, danceability <dbl>, energy <dbl>, key <dbl>,
#   loudness <dbl>, mode <dbl>, speechiness <dbl>, acousticness <dbl>,
#   instrumentalness <dbl>, liveness <dbl>, valence <dbl>, tempo <dbl>,
#   duration_ms <dbl>
```

```
newbs <- bigspotify |>
  filter(playlist_genre %in% c("rap", "rock"))

stats <- t.test(track_popularity ~ playlist_genre, data = newbs)

str(stats)
```

```
List of 10
 $ statistic : Named num 3.18
   ..- attr(*, "names")= chr "t"
 $ parameter : Named num 10236
   ..- attr(*, "names")= chr "df"
 $ p.value    : num 0.00149
 $ conf.int   : num [1:2] 0.57 2.4
   ..- attr(*, "conf.level")= num 0.95
 $ estimate   : Named num [1:2] 43.2 41.7
   ..- attr(*, "names")= chr [1:2] "mean in group rap" "mean in group rock"
 $ null.value : Named num 0
   ..- attr(*, "names")= chr "difference in means between group rap and group rock"
 $ stderr     : num 0.468
 $ alternative: chr "two.sided"
 $ method     : chr "Welch Two Sample t-test"
 $ data.name  : chr "track_popularity by playlist_genre"
 - attr(*, "class")= chr "htest"
```

```
stats[[1]]
```

```
t
3.177943
```

```
test_genre_popularity <- function(df, genre1, genre2, plot) {
  twogenresdf <- df |>
    filter(playlist_genre %in% c( {{ genre1 }}, {{ genre2 }} )) # since doing t.test must ha
```

```

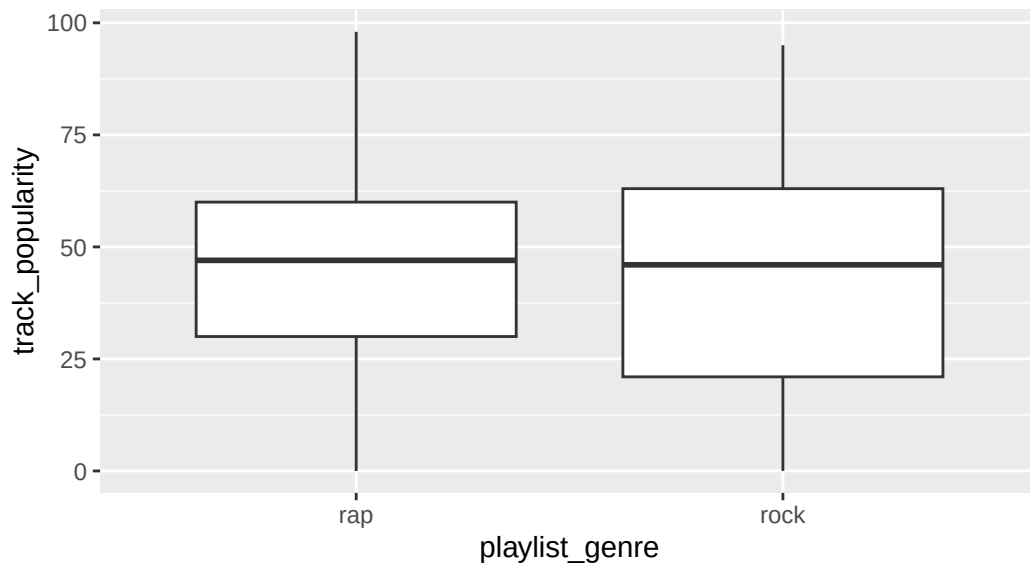
stats <- t.test(track_popularity ~ playlist_genre, data = twogenresdf) # cant pipe t.test
if (!(plot %in% c("boxplot", "histogram"))) {
  stop("Please use boxplot or histogram as the plotting option")
}
if (plot == "boxplot") {
  ggplot(twogenresdf, aes(x = playlist_genre, y = track_popularity)) +
    geom_boxplot() +
    labs(
      title = paste("Comparing", genre1, "and", genre2, "genres by popularity."),
      subtitle = paste("Two sample t-test for difference in mean popularity scores: T = ", stats$statistic)
    )
} else if (plot == "histogram") {
  ggplot(twogenresdf, aes(x = track_popularity)) +
    geom_histogram() +
    facet_wrap(~playlist_genre) +
    labs(
      title = paste("Comparing", genre1, "and", genre2, "genres by popularity."),
      subtitle = paste("Two sample t-test for difference in mean popularity scores: T = ", stats$statistic)
    )
}
}

test_genre_popularity(df = bigspotify,
                      genre1 = "rap",
                      genre2 = "rock",
                      plot = "boxplot")

```

Comparing rap and rock genres by popularity.

Two sample t-test for difference in mean popularity scores: $T = 3.177943$



```
test_genre_popularity(df = bigspotify,  
                      genre1 = "rap",  
                      genre2 = "rock",  
                      plot = "density")
```

Error in `test\_genre\_popularity()`:

! Please use boxplot or histogram as the plotting option

3. One of the harder concepts in statistics is the understanding of what the percentage in a confidence interval really means. Many introductory students believe that 95% confidence is a statement about the likelihood of the population parameter to be within the interval bounds. While not totally off-base, a better way to think about this is that if we take 1000 of different samples we expect around 950 of those samples to create 95% CIs that contain the true population parameter. Let's explore this.

Write a for loop that takes 10,000 samples of size 45 from a normal distribution with a true population mean of 25 and a true standard deviation of 4. done

Calculate the lower bound and upper bound of a 95% confidence interval for the mean using each sample. done

Check to see what proportion of simulated samples actually contain 25. (Spoiler - it should be close to 0.95)

Here are a couple hints and reminders:

- `rnorm(n = 20, mean = 0, sd = 1)` will generate a random sample of size 20 from a Normal distribution with mean 0 and standard deviation 1.
- The equation for a 95% confidence interval is the sample mean plus/minus 1.96 times sample standard deviation divided by the square root of sample size.

$$\bar{x} \pm 1.96 \times \frac{s}{\sqrt{n}}$$

- The mean of a vector of logicals will give you a proportion.

```
ci <- vector("logical", 10000)
for (i in 1:10000) {
  r_sample <- rnorm(n = 45, mean = 25, sd = 4)
  s_mean <- mean(r_sample)
  s_sd <- sd(r_sample)
  lower_ci <- s_mean - 1.96 * (s_sd / sqrt(45))
  upper_ci <- s_mean + 1.96 * (s_sd / sqrt(45))
  ci[i] <- (lower_ci <= 25) & (upper_ci >= 25)
}

mean(ci)
```

```
[1] 0.9387
```